Week 3: Branching Strategy and SCM Plan – MediAssist

Student: John Jeremy **Admission No.:** 672230 **Institution:** USIU-Africa **Date:** 21 October 2026
Repository: https://github.com/johnjeremyw/mediassist (protected `main`; example branch `feature/2-missed-dose-alert`)

Part 1 – OSS branching policy: Kubernetes

Kubernetes' CONTRIBUTING.md is deliberately short: it requires every contributor to sign the CNCF Contributor License Agreement and points to the community Contributor's Guide, which defines the actual process.

- **Branching model.** Contributors fork the repository and work on topic branches in their fork; every pull request targets the single development branch, `master`. The release team cuts long-lived `release-1.x` branches for each minor version and supports about four at a time (roughly 14 months each). Fixes land on `master` first and are then cherry-picked into release branches; cherry-picks must be merge-ready about two days before the monthly patch release.
- **Review requirements.** Review is two-phase and driven by OWNERS files in each directory. A reviewer (any org member other than the author) checks code quality and comments `/lgtm`; an approver listed in the relevant OWNERS file checks wider concerns such as API compatibility and comments `/approve`. Every affected OWNERS file needs an approver, and automation suggests at least two reviewers.
- **Merge requirements.** Nobody presses the merge button. The Prow bot's Tide component merges a PR only when it has the `lgtm` and `approved` labels, has no blocking label such as `do-not-merge/hold`, and its tests pass on a final re-run against the latest base. Squashing is optional via a label.

Lesson for MediAssist: Kubernetes needs release branches because it maintains several versions in parallel for thousands of contributors. MediAssist is a three-person team shipping one version, which Play Store users auto-update, so it can keep Kubernetes' review gate and bot-enforced checks but drop the long-lived branches.

Part 2 – SCM Plan

1. Purpose

This plan defines how MediAssist's source code, configuration and releases are identified, changed, built and released, so that every change is reviewed, tested and traceable from its issue to the version a patient installs. It turns the SCM requirements and tracking database from earlier weeks into day-to-day practice for the team.

2. Scope

In scope: the `mediassist` repository, an Expo (SDK 54) / React Native app: `App.js`, `screens/`, `components/`, `api.js`, `push.js`, `theme.js`, `assets/`, `app.json`, `package.json` and `package-lock.json`; the CI/CD and Dependabot config in `.github/`; `docs/`; and the tags, builds and Play tracks produced from them. Out of scope: the separate MediAssist-backend repository (which should adopt the same conventions), real patient data (never committed), local `.env*.local` files and personal editor settings.

3. Roles and responsibilities

Project context. Three developers (John Jeremy, Rahma and Wainaina B., all of whom have already committed), a release every two weeks, and low risk tolerance for reminder, notification, medication-data and caregiver-alert code, since a defect there can mean a missed dose.

Role	Held by	Responsibilities
Maintainer, release manager	John Jeremy (@johnjeremyw)	Owns repo settings and branch rules; triages issues; tags releases, updates the changelog, promotes builds through Play tracks, decides on rollback.
Developer, reviewer	Rahma, Wainaina B. (@WainainaB), John Jeremy	Implement issues on short-lived branches; review each other's PRs (nobody approves their own).
Code owners	John Jeremy, Wainaina B.	Must approve PRs that touch safety-critical files listed in CODEOWNERS.
Stakeholder	Course lecturer / caregiver testers	Raise requirements and feedback as issues; accept closed-beta builds.
Automation	GitHub Actions, Dependabot	Run required checks on every PR, build signed releases, open weekly dependency-update PRs.

4. Version control strategy

Tool: Git, hosted on GitHub. **Strategy: GitHub Flow** (trunk-based development with a pull-request gate).

- **Why not Git Flow:** its `develop` and `release/*` branches exist to stabilise and maintain several versions in parallel. With three developers and one live version, they would add an extra merge step and long-lived conflicts without reducing risk, and slow the two-week cadence.
- **Why not pure trunk-based (direct commits):** the first three commits went straight to `main` with no review, and one added a mis-named file (`https:/CONTRIBUTORS.md`), the kind of slip review catches. Safety-critical code needs a review and CI gate, and GitHub Flow keeps trunk-based speed while making that gate impossible to skip.
- **Branches.** `main` is the only long-lived branch and is always releasable. All work is on short-lived branches named `<type>/<issue>-<short-description>`, e.g. `feature/12-snooze-button`. Types: `feature`, `fix`, `hotfix`, `docs`, `chore`. A branch lives at most 3 days, is rebased on `main` daily, and is deleted after merging. CI rejects non-conforming names.
- **Commits and merging:** Conventional Commits (`feat(reminders): add snooze (#12)`); PRs are squash-merged so `main` keeps one commit per change and a linear history.
- **Rules on main (GitHub ruleset):** PR required with one approval from someone other than the author; code-owner approval for safety-critical files; status checks `build` and `dependency-review` pass on an up-to-date branch; conversations resolved; linear history; force pushes and deletion blocked.

5. Change control strategy

Every change starts as a GitHub issue. The maintainer triages it and classifies it into one of three classes, which decides the approval needed:

Class	Examples	Approval before merge
Standard	Report screen layout, docs, dependency bumps	Passing CI, the PR checklist and one teammate approval
Safety-critical	Reminder, add/edit medication and caregiver screens; `push.js`, `api.js`, `AuthContext.js`, `formatters.js`, `theme.js`	Impact analysis in the issue, label `safety-critical`, CI, and approval from both other developers, one a code owner (enforced by CODEOWNERS)
Emergency hotfix	Production bug causing missed or wrong reminders	Priority P1; `hotfix/` branch, CI and one code-owner approval; patch release the same day

A newcomer's first pull request:

1. Pick an issue labelled `status:ready` and assign yourself.
2. Clone, run `npm ci`, start the backend and `npx expo start`; then `git checkout main && git pull` and `git checkout -b feature/<issue>-<description>`.
3. Commit in Conventional Commit form; push with `git push -u origin <branch>`.
4. Open a PR into `main` with `Closes #<issue>` and complete the checklist (tests, accessibility, no secrets, AI-generated code declared).
5. Request a teammate's review (both, if safety-critical), fix any failing check, then squash-merge; the branch is deleted automatically.

6. CI/CD process control strategy

- **On every PR and push to main (`ci.yml`):** branch-name check; `npm ci` (exact versions from the lockfile); `npx expo install --check` (warns when packages drift from Expo SDK 54; fixed with `npx expo install --fix`); lint and tests once added; and `npx expo export` as a build check. A separate dependency-review job fails on any high or critical vulnerability. Both are required, so a red build cannot merge.
- **On a version tag (`release.yml`):** generates a CycloneDX SBOM (`npm sbom`), exports the web build, attaches both to a GitHub Release with generated notes, and starts the signed Android build on EAS Build. Release builds are never made on a laptop.
- **Secrets and dependencies:** the only CI secret is `EXPO_TOKEN`; signing keys are held by EAS and Play App Signing, and `*.jks`, `*.key` and `.env*.local` are git-ignored. Dependabot opens weekly npm and Actions update PRs, which pass the same checks.

7. Issue tracking tool and workflow

Tool: GitHub Issues with a GitHub Projects board, chosen because issues link natively to branches, PRs and commits, giving traceability for free.

- **Board columns:** Triage → Ready → In progress → In review → Done. A PR containing `Closes #n` moves its issue to Done automatically on merge.
- **Templates and labels:** bug and feature templates (the feature template includes an impact-analysis section). Labels: `type:bug|feature|docs|chore`, `priority:P1|P2|P3`, `safety-critical`, `accessibility`, `status:triage|ready`.
- **Milestones:** one per release (e.g. `v1.1.0`); whatever is not done by release day moves to the next milestone.

8. Release strategy

- **Versioning:** Semantic Versioning in `app.json` and `package.json` (currently 1.0.0); EAS auto-increments the Android `versionCode` on every build. Releases are annotated tags on `main`, not branches. Over-the-air EAS Update is not used, so every change reaches users through the tracks below.
- **Cadence:** a minor release at the end of each two-week sprint; patch releases for hotfixes as needed.
- **Procedure:** (1) close the milestone; (2) PR `chore(release): v1.1.0` bumps the version and moves CHANGELOG entries out of Unreleased; (3) after merge, `git tag -a v1.1.0 -m "v1.1.0" && git push origin v1.1.0`; (4) CI builds the release; (5) `eas submit` to Play internal testing → closed testing with caregiver testers → staged production rollout 10% → 50% → 100%, watching crash rate in Android vitals between steps.
- **Rollback:** halt the staged rollout in Play Console, then ship a `hotfix/` branch as the next patch version. Play cannot downgrade a `versionCode`, so recovery is always forward, from the last good tag.

Part 3 – Repository

https://github.com/johnjeremyw/mediassist. The repository matches this plan: `main` is protected by the rules in Section 4; `CONTRIBUTING.md` repeats the branch convention; `.github/` holds CODEOWNERS, the PR and issue templates, `ci.yml`, `release.yml` and Dependabot; these were added through the first reviewed PR, from `docs/1-scm-plan`, and `feature/2-missed-dose-alert` shows an active short-lived branch.

References

Kubernetes. (n.d.). CONTRIBUTING.md. GitHub.
https://github.com/kubernetes/kubernetes/blob/master/CONTRIBUTING.md

Kubernetes Contributors. (n.d.). OWNERS files. Kubernetes Contributors Site.
https://www.kubernetes.dev/docs/guide/owners/

Kubernetes Contributors. (2022, December 12). Prow and Tide for Kubernetes contributors.
https://www.kubernetes.dev/blog/2022/12/12/prow-and-tide-for-kubernetes-contributors

Kubernetes. (2026). Patch releases. https://kubernetes.io/releases/patch-releases/